arXiv:2105.11766v3 [quant-ph] 24 Jun 2022

An evolving objective function for improved variational quantum optimisation

Ioannis Kolotouros* and Petros Wallden†
University of Edinburgh, School of Informatics, EH8 9AB Edinburgh, United Kingdom
(Dated: June 27, 2022)

A promising approach to useful computational quantum advantage is to use variational quantum algorithms for optimisation problems. Crucial for the performance of these algorithms is to ensure that the algorithm converges with high probability to a near-optimal solution in a small time. In Barkoutsos et al [Quantum 2020] an alternative class of objective functions, called Conditional Value-at-Risk (CVaR), was introduced and it was shown that they perform better than standard objective functions. Here we extend that work by introducing an evolving objective function, which we call Ascending-CVaR and that can be used for any optimisation problem. We test our proposed objective function, in an emulation environment, using as case-studies three different optimisation problems: Max-Cut, Number Partitioning and Portfolio Optimisation. We examine multiple instances of different sizes and analyse the performance using the Variational Quantum Eigensolver (VQE) with hardware-efficient ansatz and the Quantum Approximate Optimization Algorithm (QAOA). We show that Ascending-CVaR in all cases performs better than standard objective functions or the "constant" CVaR of Barkoutsos et al [Quantum 2020] and that it can be used as a heuristic for avoiding sub-optimal minima. Our proposal achieves higher overlap with the ideal state in all problems, whether we consider easy or hard instances – on average it gives up to ten times greater overlap at Portfolio Optimisation and Number Partitioning, while it gives an 80% improvement at Max-Cut. In the hard instances we consider, for the number partitioning problem, standard objective functions fail to find the correct solution in almost all cases, CVaR finds the correct solution at 60% of the cases, while Ascending-CVaR finds the correct solution in 95% of the cases.

I. INTRODUCTION

We have recently entered the era where quantum computers have scaled up, from small proof-of-principle devices to devices that are beyond the classical simulation limit opening the prospect for providing computational speed-ups. However, we are still very far from the point that large fault tolerant quantum computers are developed. Our period has been termed as Noisy Intermediate Scale Quantum (NISQ) device era [1] and refers to the time that the existing devices vary from ≈ 50 qubits of Google's quantum-advantage[1] experiment [2] to devices with $O(1000)$ qubits that are anticipated in a horizon of five to ten years.

There are two paths forward for quantum computing. The "long-term" path requires to intensify the efforts (theoretical and experimental) to overcome existing barriers and truly scale up these devices to the large fault-tolerant regime. The "near-term" one, is to determine if and how these NISQ devices can be used directly and offer advantage for problems of practical importance. A promising approach in the latter path, is the use of hybrid quantum-classical algorithms. A leading class of candidate algorithms, both due the possible importance of the applications and the promise it shows, is the class of variational quantum algorithms for optimisation problems.

One can divide variational quantum algorithms (see more details II]) into three main steps. The first step is to map the targeted problem to the mathematical task that these algorithms are designed to solve, which is the search for the ground state energy of a Hamiltonian[2]. The second step, is a method to estimate the energy of a quantum state, given a (polynomial in the size of the input) number of copies. Finally, the third step consists of a parameterised family of quantum states ("ansatz") and a classical optimiser that given the above tools, outputs efficiently an approximation of the ground state energy. This is done by finding the choice of parameters that lead to the quantum state that has the smallest energy.

The success of the algorithms depend on all those steps and extensive research on improving each of them exists, indicatively, [3] used warm-starting to improve QAOA on low depth, [4] improved QAOA by introducing a non-local version which outperformed classical QAOA on 3-regular graphs, [5, 6] introduced different procedures on how to optimise the variational parameters and [7] used reinforcement learning to assist the

* i.kolotouros@sms.ed.ac.uk
† petros.wallden@ed.ac.uk
[1] Also known as "quantum computational supremacy".
[2] Mathematically this is simply evaluating the smallest eigenvalue of a Hermitian matrix.

classical optimisation.

What we focus in this contribution is the third part, and specifically on how to use the measurement outcomes performed in estimating the energy of a quantum state to (i) accelerate the speed and (ii) improve the accuracy that the classical optimiser finds an (approximation of the) ground state and thus solves the problem optimally. Prior to our work, [8] inspired by statistical physics, used a Gibbs objective function to improve the performance. Minimising the *infidelity* between the parameterised state and a target state [9, 10] appears to be another promising approach.

For classical optimisation problems, the solution (ground state) is one of the computational basis quantum states. Preparing a quantum state that has big overlap with that state is sufficient to give a good and quick approximation of the ground state. For example, if one can achieve a constant but possibly small overlap with the correct solution, it is guaranteed after sampling this state a constant number of times to obtain at least one sample of the true ground state. In [11], the authors used this idea, and instead of evaluating the proximity of a quantum state to the desired (ground state) by minimising the (overall) energy, aimed to minimise the energy of the lowest tail of a quantum state. This, intuitively, would succeed quicker in finding a quantum state that has a non-negligible overlap with the solution (but not necessarily very high overlap). This state, however, suffices to solve the problem. This intuition was also confirmed with numerical simulations. In other words, the cost function used in the classical optimiser, in order to find the optimal parameters, was not the energy of the quantum state, but the tail of the corresponding distribution.

Inspired by this idea but also by adiabatic quantum computing [12], we consider here an evolving cost function. In our proposal the way that the cost of a quantum state is computed, dynamically changes during the classical optimisation process. We start with a cost function as in [11] focusing on a small tail, but during the optimisation process we gradually increase the tail (fraction of the distribution we "count") until we reach a point that all the distribution is included i.e. we measure the full expectation value of the energy (as in "standard" cost functions).

Our contributions.

- We introduce an evolving objective function that starts with the CVaR defined in [11] and gradually in the optimisation process becomes the full energy of the quantum state. Alternative forms of this Ascending-CVaR objective functions are considered and a linear and a sigmoid functions (that appear to perform better) are selected.
- We test our proposal, with classical numerical simulations (using up to 20 qubits), both in the setting of VQE with hardware efficient ansatz and in QAOA. Our results suggest that our proposal leads to faster convergence with bigger overlap with the ideal solution than prior works, while crucially, succeeds in obtaining the solution in (many) instances that other techniques fail altogether (see Section VII for statistics and comparisons).
- Our analysis is done for three different combinatorial optimisation problems, namely Max-Cut, Number Partitioning and Portfolio Optimisation. We consider many different instances and problem sizes where the conclusions persist in all cases. This has importance in its own right, since these problems are important by themselves, and our proposal gives an approach to improve the performance and bring closer achieving "useful" quantum advantage. Interestingly, our method offered greater advantage in "hard instances" of the problems, where the other methods frequently failed to find the solutions altogether.

Structure. In Section II we give the essential background: We introduce the variational quantum algorithms and specifically, the variational quantum eigensolver and the quantum approximate optimisation algorithm. We then introduce the CVaR objective function of [11] and finally analyse the three different combinatorial optimisation problems that we use as case-studies. In Section III we introduce our novel method called *Ascending-CVaR* and we discuss the hyperparameters of our model. In section IV we illustrate our method using a small instance as an example. In Section V we discuss our methodology. In section VI we present the results of our method compared to existing objective functions. We conclude in Section VII with a general discussion of our method and future work.

II. PRELIMINARIES

We introduce two of the main Variational Quantum Algorithms [13], the CVaR objective function [11] and the three types of combinatorial optimisation problems that we use.

A. Variational Quantum Algorithms

Here we revise the methods with focus on optimisation problems. The general framework of a variational quantum algorithm is outlined in Figure 1. The first step is to map the classical cost function $C(\boldsymbol{x})$ that describes the optimisation problem, into an interacting qubit Hamiltonian H_C whose ground state gives the solution we are seeking.

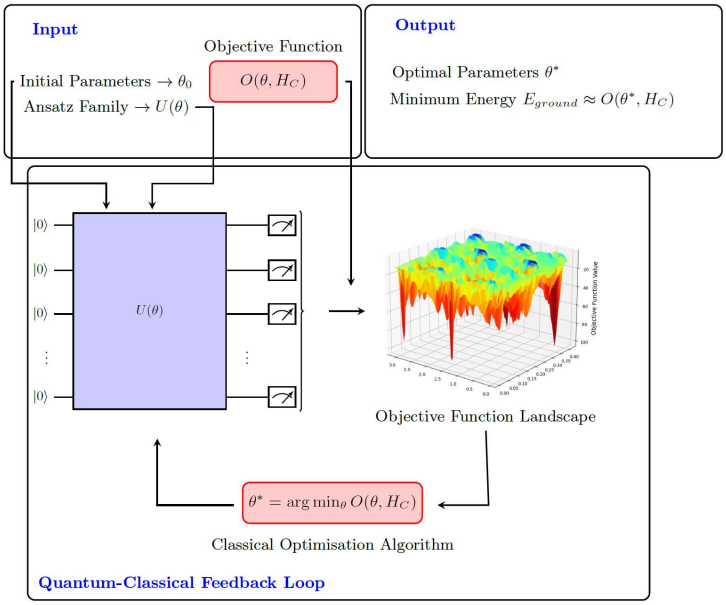

Figure 1: General framework of a variational quantum algorithm. The optimisation problem, described by a cost function $C(\boldsymbol{x})$ is mapped to an interacting qubit Hamiltonian H_C. A parameterised family of states ("ansatz") $U(\boldsymbol{\theta})$ with random initial parameters is chosen and a quantum-classical feedback loop iteratively updates the parameters $\boldsymbol{\theta}$. The optimisation ends when the stopping condition is met, and the optimal parameters $\boldsymbol{\theta}^*$ are outputted.

The second step is to choose an ansatz family of unitary operators $U(\boldsymbol{\theta})$. This family is both efficiently expressible and trainable, parameterised by a μ-parameter vector $\boldsymbol{\theta} = (\theta_1, ..., \theta_\mu)$ where $\mu = \mathcal{O}(poly(n))$ and n is the system size. In general, the parameters are initialized at random[3]. The third step is to evaluate some objective function, usually taken to be the expectation value of the problem's Hamiltonian on the state considered $\langle\psi(\boldsymbol{\theta})| H_C |\psi(\boldsymbol{\theta})\rangle$. This is done by preparing the state (applying the unitary $U(\boldsymbol{\theta})$ on the initial state) and then measuring the output in the computational basis and repeating this procedure for a given number of times (typically called "shots"). This number determines the accuracy the objective function is evaluated. The fourth step is to update the parameters and repeat step three, iteratively using some classical optimiser until a stopping condition is satisfied. We then say that the parameters are optimal, i.e.

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} O(\boldsymbol{\theta}, H_C) \tag{1}$$

The state produced by these parameters, $|\psi(\boldsymbol{\theta}^*)\rangle = U(\boldsymbol{\theta}^*)|0\rangle^{\otimes n}$, can be used to give an estimate of the ground state energy of the Hamiltonian H_C and thus an approximate solution to the desired optimisation problem. The objective function used during this process, as stated above, typically coincides with the expectation value of the problem's Hamiltonian. However, we note here that other choices may also be possible, especially if we realise that the true target of the optimisation algorithm is to sample, at least once, the optimal solution. This can efficiently be produced if the output state has a sufficiently large (or more precisely simply non-vanishing) overlap with the optimal solution $|\langle\psi(\boldsymbol{\theta}^*)|\psi_{opt}\rangle|^2$.

[3] There are cases that a "clever" initialization could lead to faster convergence [14, 15]

B. Conditional Value-At-Risk

Barkoutsos et al. [11] used an alternative objective function. They demonstrated that their proposal performed better than minimising the expectation value. The key observation is that for optimisation problems the optimal solution is a computational basis state. For computational basis states, one can compute their energy (efficiently). For a general quantum state $|\psi(\boldsymbol{\theta})\rangle$ one can prepare and measure it (multiple times) in the computational basis, and the expectation value of the energy is simply the average of the individual computational basis states energies. To find the overlap of

this state with the optimal solution (ground state) one can simply observe the frequency of the computational basis state with the smallest energy. Naturally, if that overlap is too small (or even zero), it is possible that none of the measurements outcomes will give the solution. On the other hand it is also clear that the overlap of this state with computational basis vectors with high energy are irrelevant for finding the ground state. The idea of [11] was to use this observation and instead of using all the measurement outcomes and compute the expectation value, they used as objective function the lower tail of the distribution of energies obtained, i.e. ignored all but a small fraction (with smallest energy) of their measurement outcomes.

They then demonstrated that their technique succeeded in getting quicker a quantum state that has a sufficiently large overlap with the ground state. This in turn, is sufficient to actually find this ground state, since as a final step, once the optimal $\boldsymbol{\theta}^*$ is found, one can keep the computational vector that has the smallest energy only. Specifically, let H_k be the energy corresponding to a computational basis vector, and let us order them in such a way that larger k corresponds to larger energy. For each state, one repeats the measurement K-times, so there are (up to) K distinct values H_k. In [11] a new parameter α was introduced. Let $\alpha \in (0, 1]$ be the fraction (part of the tail) that we want to keep. This fraction, typically, needs to be non-negligible (we can assume for simplicity, that is constant). Then the objective function that was used, was the average of the smallest αK samples, i.e.

$$CVaR_\alpha = \frac{1}{\lceil \alpha K \rceil} \sum_{k=0}^{\lceil \alpha K \rceil} H_k \tag{2}$$

In order to achieve the same accuracy when evaluating this objective function, as the accuracy achieved when computing the expectation value using K shots, it is clear that the number of runs of the preparation circuit need to be increased to K/α.

As it was proven by [11], the angles $\boldsymbol{\theta}^*$ that minimise CVaR_α do not (in general) correspond to minima of the expectation value. As a result, the angles that lead to the smallest possible α-tail differ from the angles that minimise the average of the samples. This fact motivates to introduce a lower α-tail optimisation so as to achieve an overlap with the optimal state of at least α, i.e find optimal $\boldsymbol{\theta}^*$ that satisfy:

$$|\langle \psi(\boldsymbol{\theta}^*) | \psi_{opt} \rangle|^2 \geq \alpha \tag{3}$$

C. Combinatorial Optimisation Problems

We test our proposed method in various instances of three different combinatorial optimisation problems. These are all important problems in their own right, so improving the performance of variational quantum algorithms for these problems is of independent interest. Moreover, testing our proposed objective function on different types of combinatorial optimisation problems demonstrates that improvements observed are generic and motivates further use for different applications. Given that our proposal's starting point is the work of [11], we included the problems that they tested their proposal to allow for more direct comparison.

The easiest way to use variational quantum algorithms for an optimisation problem is to first map the problem to a *Quadratic Unconstrained Binary Optimization* (QUBO) problem. This is what we will do for all our examples. QUBO problems seek to solve (find the $\boldsymbol{x}$ that minimises the expression):

$$\min_{\boldsymbol{x}} \left(b^T \boldsymbol{x} + \boldsymbol{x}^T A \boldsymbol{x} \right) \tag{4}$$

where $b \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. These cost functions can easily be mapped to an Ising Hamiltonian [16] by first transforming the binary variables $x_i \in \{0, 1\}$ according to:

$$x_i = \frac{1 - z_i}{2} \tag{5}$$

where $z_i \in \{-1, +1\}$ are spin variables, and then turning the cost function to a Hamiltonian by promoting these variables to Pauli σ_i^z operators, one for each qubit i. The QUBO problem then transforms to

$$\min_{z} c^T \boldsymbol{z} + \boldsymbol{z}^T Q \boldsymbol{z} \tag{6}$$

where the new $c \in \mathbb{R}^n$ and $Q \in \mathbb{R}^{n \times n}$ are easily computable.

Then, by replacing the spin variable z_i with the Pauli σ_i^z operator with corresponding eigenvalues $\{-1, +1\}$, the problem translates into finding the ground state, i.e. the spin configuration, of an n-qubit system interacting with the Hamiltonian:

$$H = \sum_{i=1}^{n} c_i \sigma_i^z + \sum_{i=1}^{n} Q_{ij} \sigma_i^z \sigma_j^z \tag{7}$$

1. *Max-Cut Problem*

The first problem is *Max-Cut*. It is one of the most studied combinatorial problems in the context of variational quantum algorithms due to the simplicity and

guaranteed performance at least for some instances [17, 18].

Let $G(V, E)$ be a non-directed n-vertex graph, where V is the set of vertices, E is the set of edges, and w_{ij} are the weights of the edges. A *cut* is defined as a bipartion of the set V into two disjoint subsets P, Q, i.e. $P \cup Q = V$ and $P \cap Q = \emptyset$. Equivalently, we label every vertex with either 0 or 1, where it is understood that the vertex belongs to set P if it takes the value 0 and to set Q if it takes the value 1. The aim is to maximise the following cost function:

$$C(\boldsymbol{x}) = \sum_{i,j=1}^{n} w_{ij} x_i \left(1 - x_j\right) \tag{8}$$

This intuitively corresponds to finding a partition of the vertices into two disjoint sets that "cuts" the maximum number of edges. By applying the transformation, Eq. (5), the cost function transforms into:

$$C(\boldsymbol{z}) = \sum_{\langle i,j \rangle \in E} \frac{w_{ij}}{2} \left(1 - z_i z_j\right) \tag{9}$$

Maximising the cost function above corresponds into finding the ground state of the Hamiltonian[4]:

$$H_C = - \sum_{\langle i,j \rangle \in E} \frac{w_{ij}}{2} \left(1 - \sigma_i^z \sigma_j^z\right) \tag{10}$$

Max-Cut is known to be NP-Hard. The best classical approximation algorithm is that of Goemans and Williamson which uses semi-definite programming to achieve an approximation ratio, Eq. (A4), $r^* \approx 0.87856$ for all graphs. Note, that being NP-Hard implies that we do not expect to have an efficient quantum algorithm (poly-time) to solve the problem for its hardest instances[5], but we could definitely get improvements using quantum algorithms (either by smaller speed-ups or by heuristics that could solve more instances than classical heuristics).

Although it was proven that constant-depth QAOA does not outperform GW for certain class of problems [4], there are instances where the approximation ratio of the former is larger than the latter [19]. Note here that QAOA beats random guessing even at $p = 1$ [17], while Machine Learning techniques have been used to classify for which graph types is better to use QAOA instead of GW [20]. In general, however, the performance of QAOA in intermediate depths is still highly unexplored.

[4] Note the overall minus sign that turns the maximisation of the cost function to finding the minimum energy for the Hamiltonian.

[5] NP is strongly believed to not be included in BQP

2. Number Partitioning

The second problem is *Number Partitioning* and is stated as follows. Given a set of N positive integers $S = \{n_1, n_2, ..., n_N\}$, the target is to find a bipartion of the set S into two disjoint subsets P, Q, where $P \cup Q = S$ and $P \cap Q = \emptyset$ so that the difference between the sum of the elements on the set P and the set Q is minimized. We thus want to minimize the cost function:

$$C(\boldsymbol{x}) = \left(\sum_{i=1}^{N} (2x_i - 1) n_i \right)^2 \tag{11}$$

The binary string $\boldsymbol{x} = x_1 x_2 \ldots x_n$ corresponds to one configuration where a number n_i is placed in the P set ($x_i = 0$) or in the Q set ($x_i = 1$). The cost function can easily be mapped to the Ising Hamiltonian:

$$H_C = \left(\sum_{i=1}^{N} \sigma_i^z n_i \right)^2 \tag{12}$$

By expanding the cost function Eq. (12), the cost function can be written as:

$$H_C = \sum_{i \neq j} (n_i n_j) \sigma_i^z \sigma_j^z + \sum_{i=1}^{N} n_i^2 \tag{13}$$

If we neglect the constant term, we can see that the Number Partitioning problem can be easily mapped to the *Sherrington-Kirkpatrick model* which is an energy minimization problem with an all-to-all random couplings which was recently analysed on [21].

Although the problem is known to be NP-Hard, it is also known as the "easiest hard problem". That is, because there exists a "hard-easy" phase transition [22] where instances belonging in the easy-phase can be efficiently tackled using heuristics [23]. Interestingly, it appears that one may be able to tackle some of the instances in the "hard phase" using variational quantum algorithms.

3. Portfolio Optimisation

The third problem is *Portfolio Optimisation* [24, 25] and is stated as follows. Given a set of n assets $\{0, \cdots, n\}$, corresponding expected returns μ_i and covariances Σ_{ij}, a risk factor $q > 0$ and a budget $B \in \{1, \ldots, n\}$, the considered portfolio optimisation problem tries to find a subset of assets $P \subset \{1, \ldots, n\}$ with $|P| = B$ such that the resulting *q-weighted-mean-variance*, i.e. $\sum_{i \in P} \mu_i - q \sum_{i,j \in P} \Sigma_{ij}$, is maximised. In other words, we want to maximise the cost function:

$$C(\boldsymbol{x}) = \sum_{i=1}^{n} \mu_i x_i - q \sum_{i,j=1}^{n} \Sigma_{ij} x_i x_j \tag{14}$$

along with the constraint

$$\sum_{i=1}^{n} x_i = B \tag{15}$$

The *portfolio vector* $x \in \{0,1\}^n$, consisting of n binary decision variables, indicates whether an asset is picked ($x_i = 1$) or not ($x_i = 0$). The constraint in (15) is translated as an extra penalty term in the Hamiltonian $(\sum_{i=1}^{n} x_i - B)^2$.

The problem is known to be NP-complete [26]. We apply the transformation, Eq. (5), so the cost function transforms into:

$$C(\boldsymbol{z}) = -q \sum_{i,j=1}^{n} \frac{\Sigma_{ij}}{4} z_i z_j + \sum_{i=1}^{n} \left(\sum_{j=1}^{n} \frac{q\Sigma_{ij} z_i}{2} - \frac{\mu_i z_i}{2} \right) + \sum_{i=1}^{n} \left(\frac{\mu_i}{2} - \sum_{j=1}^{n} \frac{q\Sigma_{ij}}{4} \right) \tag{16}$$

which, along with the extra penalty term, corresponds to minimising the Hamiltonian :

$$H_C = \sum_{i,j=1}^{n} \frac{q\Sigma_{ij}}{4} \sigma_i^z \sigma_j^z - \sum_{i=1}^{n} \left(\sum_{j=1}^{n} \frac{q\Sigma_{ij}\sigma_i^z}{2} - \frac{\mu_i \sigma_i^z}{2} \right) - \sum_{i=1}^{n} \left(\frac{\mu_i}{2} - \sum_{j=1}^{n} \frac{q\Sigma_{ij}}{4} \right) + \left(\sum_{i=1}^{n} \sigma_i^z + \frac{n}{2} - B \right)^2 \tag{17}$$

Portfolio optimisation as given in Eq. (14) was recently tackled using variational quantum algorithms [27], using warm-starting QAOA [3] and on D-wave systems using quantum annealing [28]. Prior to our work, [29] developed a quantum-walk-based optimisation algorithm and [30] considered a more general setting of portfolio optimisation, called dynamic portfolio optimisation, where one has to allocate weights to a number of assets in a period of time in order to maximise the overall return.

III. ASCENDING-CVAR

The CVaR cost function of [11] was shown to perform better in general, than the "standard" expectation value. There are three observations, however, that motivates our proposal. First, as noted in [11], the choice of α is somehow random, and importantly, for different problems and even for different instances of the same class of problems, the optimal choice of α varies in a non-obvious (e.g. monotonic) way. The performance of the algorithm's speed, but also if it finds the solution at all, depends on that choice. The second point is that optimising with a fixed small α has further disadvantages: (i) it "finds" parameters $\boldsymbol{\theta}$ that result to a state that does not have the greatest overlap with the solution and (ii) the true running time of the algorithm to achieve same accuracy is larger, in other words for each iteration one requires $1/\alpha$ times more measurements to achieve the same accuracy in estimating the cost function (since only the lower α fraction of the measurements are used). Finally, the third observation is that the $CVaR_\alpha$ objective functions with different α have a different energy landscape. For any fixed choice of α the optimiser could "get stuck" at a local minimum. Interestingly, if one varies the α during the optimisation, while we still ensure that if the algorithm finds the true ground state it remains there, we also avoid getting stuck at local minima since those are different for different choices of α. Therefore if the optimiser reaches a point that has a local minimum for one value of α, when α changes this point (may) no longer be a local minimum and thus could continue "moving" towards the true global minimum (ground state).

Let's say that an optimisation problem has an optimal solution which is a computational basis state and we denote it as $|\psi_{opt}\rangle$. Let's also assume that a parameterised family of gates, $U(\boldsymbol{\theta})$, acts on the $|0\rangle^{\otimes n}$ state and produces the state

$$|\psi(\boldsymbol{\theta})\rangle = a_{opt}(\boldsymbol{\theta}) |\psi_{opt}\rangle + (1 - a_{opt}(\boldsymbol{\theta})) |\psi_{other}\rangle \tag{18}$$

where $|\psi_{other}\rangle$ is the superposition of all sub-optimal computational basis states. Let's also assume that this parameterised family of states can achieve *a maximum overlap κ with the optimal solution*[6]. We can write the state $|\psi\rangle$, corresponding to the state with the highest overlap, without loss of generality as:

$$|\psi\rangle = \sqrt{\kappa} |\psi_{opt}\rangle + \left(1 - \sqrt{\kappa}\right) |\psi_{other}\rangle \tag{19}$$

Proposition 1. *For the family of states in Eq.* (18) *and for all $\alpha \leq \kappa$:*

$$\min_{\boldsymbol{\theta}} CVaR_\alpha(\boldsymbol{\theta}) = \min_{|\phi\rangle} \langle\phi| H_C |\phi\rangle \tag{20}$$

i.e. all $CVaR_\alpha$ with $\alpha \leq \kappa$ share the same minimum objective function value which is the smallest eigenvalue of the Hamiltonian.

[6] In other words, the complex coefficient $a_{opt}(\boldsymbol{\theta})$ corresponding to the probability of sampling the optimal solution $Prob(opt) = |a_{opt}|^2$ has a maximum value : $\max_{\boldsymbol{\theta}} |a_{opt}(\boldsymbol{\theta})|^2 = \kappa$)

It is clear from Proposition 1 that *all* $CVaR_\alpha(\boldsymbol{\theta})$ *with* $\alpha \leq \kappa$ *share the same ground state*, which is the true optimum of the optimisation problem. Thus all angles $\boldsymbol{\theta^*}$ that correspond to a global minimum of $CVaR_{\alpha_1}$ will also correspond to a global minimum of $CVaR_{\alpha_2}$ if $\alpha_2 \leq \alpha_1 \leq \kappa$. For example for an ansatz $U(\theta)$ that is able to attain 10% overlap with the optimal computational basis state, if one is able to find the global minimum of $CVaR_{0.1}$, which means that 10% of the measurements correspond to the ground state, then it is clear that all $\alpha < 0.1$ will also be minimised by the same angles.

Proposition 2. *Let an optimisation problem with an optimal solution* $|\psi_{opt}\rangle$ *corresponding to a computational basis state. For any parameterised family of gates* $U(\boldsymbol{\theta})$ *that can achieve a maximum overlap* κ *with the optimal solution, the angles* θ^* *that correspond to the global minimum of* $CVaR_{\alpha_1}$ *will also correspond to a global minimum for* $CVaR_{\alpha_2}$ *if* $\alpha_1 \leq \alpha_2 \leq \kappa$. *The converse does not necessarily hold.*

In other words, Proposition 2 states that if $\boldsymbol{\theta^*} = \arg\min_{\boldsymbol{\theta}} CVaR_{\alpha_2}(\boldsymbol{\theta})$ then also $\boldsymbol{\theta^*} = \arg\min_{\boldsymbol{\theta}} CVaR_{\alpha_1}(\boldsymbol{\theta})$ for all $\alpha_1 \leq \alpha_2 \leq \kappa$. This indicates why decreasing α may not seem like a good choice. If for example the optimiser is able to find the optimal angles that minimise $CVaR_{\alpha_1}$ with $\alpha_1 \leq \kappa$, then for all $\alpha_2 < \alpha_1$ they will still remain optimal angles and thus will not be able to achieve a higher overlap state.

Proposition 3. *A local minimum for* $CVaR_{\alpha_1}$ *does not necessarily correspond to a local minimum for* $CVaR_{\alpha_2}$ *if* $\alpha_1 \neq \alpha_2$.

Proposition 3 was proven using a counterexample in [11]. All these Propositions are important for introducing a non-stationary optimisation technique that avoids local minima. We know from Proposition 1 that all $CVaR_\alpha$ objective functions with $\alpha \in (0, \kappa]$ share the same minimum objective value which is the ground state energy of the Hamiltonian. We also know from Proposition 2 that many of the global minima for α_1 may not be a global minimum for α_2 if $\alpha_1 < \alpha_2$ and thus increasing α introduces extra information about the optimality of states. Finally, Proposition 3 indicates that different objective functions are associated with different energy landscapes as they do not agree on the local minima.

However, knowing the maximum overlap κ in advance is not always possible. In the case of VQE with a hardware efficient ansatz, it can be shown that $\kappa = 1$ and so $\min_{\boldsymbol{\theta}} CVaR_\alpha(\boldsymbol{\theta}) = \min_{|\phi\rangle} \langle\phi| H_C |\phi\rangle$ for every $\alpha \in (0, 1]$. On the other hand for the QAOA ansatz, our experiments showed that κ is usually small on low depth but increases with the number of layers

The cost functions used in variational quantum algorithms, to our knowledge, are "constant in time", meaning that the whole optimisation is run with a fixed cost function. To solve the issue of "selecting the best α" and the other reasons listed above, we propose to use a dynamically evolving cost function, that essentially passes through a fixed set of α values. In the case of VQE, it is initialised in a very small value and the optimisation ends with $\alpha = 1$ that is the standard expectation value of the Hamiltonian. We call all these cost functions *Ascending-CVaR*. This has also a great(er) number of free choices, since we can now freely choose the (ascending) function. However, all choices we tried for the ascending function performed (in general) better than fixed α, which indicates that the evolving cost function is a promising approach. For the remaining of the paper we focused on two functions that performed better:

The *linear ascending* in which the parameter α_t is iteratively and discretely increased by the rule:

$$\begin{aligned} \alpha_{t+1} &= \alpha_t + \lambda \\ CVaR_{\alpha_t} &= \frac{1}{\lceil \alpha_t K \rceil} \sum_{k=0}^{\lceil \alpha_t K \rceil} H_k \end{aligned} \tag{21}$$

where $\lambda \in [0.025, 0.045]$ is the *ascending factor* and $0 < \alpha_t \leq 1$.

The *sigmoid ascending* in which the parameter α_t is discretely increased according to the function :

$$\alpha_t = \frac{1}{1 + e^{5-\lambda t}} \tag{22}$$

where $\lambda \in [0.3, 0.4]$ is again the ascending factor and $0 < \alpha_t \leq 1$.

To reach this conclusion we tested four different functions, a *sigmoid*, a *linear*, an *exponential* and a *logarithmic* (see Figure 2) on VQE-CVaR$_{\alpha_t}$ with various different ascending rates. All functions were tested on all three problems. The metrics used were the magnitude of the overlap with the optimal solution, the success rate (i.e. the number of times where it succeeds to achieve a non-negligible overlap) as well as the average time taken to achieve at least 10% overlap (for details see V).

The linear ascending, Eq. (21), and the sigmoid ascending, Eq. (22), functions have the most steady behavior as it can be seen in Figure 3, outperforming the other two types on the majority of instances. The sigmoid was slightly slower in terms of speed, which is why we mainly used the linear one. However, as we will discuss in the next section, it appears that it may be better in some classes of problems especially on harder instances with ~ 50 qubits. It seems that in those cases, the optimiser is doing better spending the majority of its iterations on low α values and thus the sigmoid performs better.

It is worth noting that increasing α to $\alpha = 1$ where it becomes the expectation value is not necessary. The

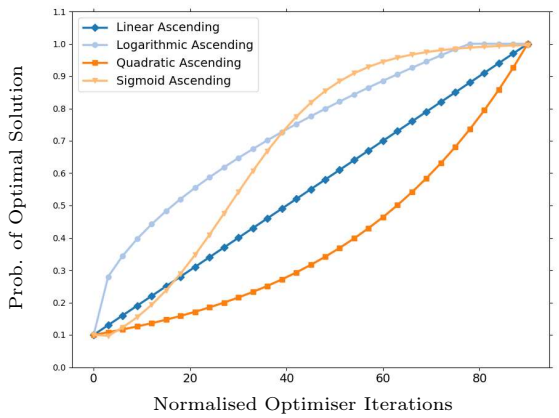

Figure 2: Different choices for the ascending function. All functions start from the same initial point, $\alpha_0 = 0.01$ and ascend until $\alpha_f = 1$ is reached.

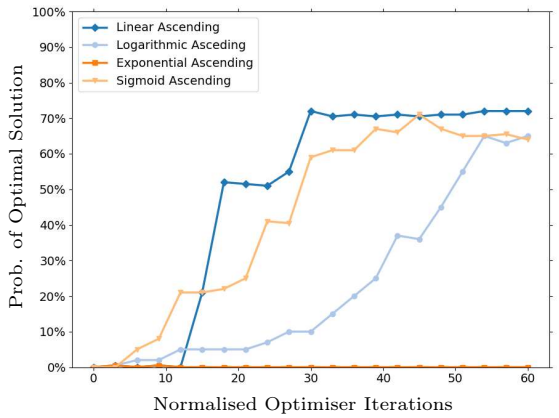

Figure 3: Portfolio optimisation instance for 18 assets and different ascending functions. The blue line (down-pointing marker) indicates the linear ascending and always achieves a high overlap with the optimal solution in contrast to the orange line (line marker), the exponential ascending, which fails in almost any instance.

whole point of variational algorithms is to achieve a constant non-negligible overlap with the optimal or near-optimal solution. For that reason one could only vary α until it reaches a threshold ϵ truncating the optimisation and reducing the number of iterations by a considerable amount.

We remark here that *Ascending-CVaR* is fundamentally different from an adaptive strategy that selects the optimal value for the parameter α. Specifically, by looking at the results in Section VI we can see that our method is able to reach quantum states that result in a very high overlap with the optimal state (almost equal to unity) that no constant choice of α would be able to attain. Even when the question is whether we find the solution with at least some small probability, Ascending-CVaR succeeds in cases where all of the fixed α failed.

In order to get the intuition why our method works one should think of what varying α actually does. The optimiser, at each iteration, moves towards a local (or a global) minimum corresponding to the instantaneous value of α. By increasing α at every step, the optimisation is able to "see" a larger part of the energy distribution of the quantum state. This translates to the optimiser gaining additional information about the quantum state which modifies the objective function landscape.

This extra information alters the landscape and is thus able to "erase" false local minima, while using the previous step as "initialisation" it is unlikely that the optimiser will get stuck in new sub-optimal minima. Moreover, since only the global minima are invariant under α transformations, this change in the landscape will not affect any correct moves of the optimiser. In other words, if the optimisation algorithm did converge in a sub-optimal local minimum for a value of α, it may not still be in a local minimum by switching into a different value of α. Hence, the *Ascending-CVaR* algorithm is guiding the trajectory of the optimiser in the highly parameterised space until it reaches a (nearly) optimal solution.

Finally, the reason our method is sensitive to the choice of the ascending factor λ is related to the speed that the optimiser is receiving this extra information (see Appendix C on how we numerically test the ascending factors for the different optimisation problems).

The pseudocode for the Ascending-CVaR algorithm is outlined in Algorithm 1

IV. WHY OUR METHOD WORKS: AN EXAMPLE

It would be illustrative to describe how local minima may vanish when the objective function is changed during the optimisation. In Figure 4 we plot the $CVaR_\alpha$ objective function landscape for different values of α. We choose to draw the landscape for the QAOA algorithm with depth $p = 1$ because the two parameters β, γ make it suitable to visualise in a 2D plot. On the contrary, VQE with a hardware efficient ansatz even on depth $p = 1$ would require $2n$ parameters.

It can be easily seen that the positions of the local minima change but the position of the true global minimum remain the same while the condition $\alpha \leq \kappa$ holds (see Proposition 1). However, in order to make it clearer

Algorithm 1: General Ascending-CVaR Optimisation Algorithm

Require: Cost Function $C(\boldsymbol{\theta})$;
$\boldsymbol{\theta^{(0)}} \leftarrow$ Random initial parameters in the domain of $C(\boldsymbol{\theta})$;
$\alpha_0 \leftarrow$ Initial α;
$g(\alpha) \leftarrow$ Ascending function;
$U(\boldsymbol{\theta}) \leftarrow$ Ansatz Family
for $i = 1, 2, \ldots$ **do**
  $\boldsymbol{\theta^*} = \arg\min_{\boldsymbol{\theta}} CVaR_{\alpha_{i-1}}(\boldsymbol{\theta})$ with initial parameters $\boldsymbol{\theta^{(0)}}$;
  if *stopping condition is met* **then**
    return $\boldsymbol{\theta^*}$;
  end
  $\alpha_i \leftarrow g(\alpha_{i-1})$;
  $\boldsymbol{\theta^{(0)}} = \boldsymbol{\theta^*}$;
end

for the reader, we choose to circle the position of a local minimum, located at $\gamma = 0.15, \beta = 1.75$. In this case, we can see how the local minima vanishes during the variation of the objective function. An optimiser that could stuck during the optimisation on a fixed value of α could "unstuck" with the change of α.

The problem corresponding to the figures is a small instance of the Number Partitioning problem with size $n = 8$. Even in a small size instance like this, the landscape is full of sub-optimal local minima where the optimiser could falsely converge to. This case-problem however does not constitute an example to prove the value of our method but only to visualise the changes in the energy landscape. The biggest improvements were observed in high dimensional expressive parameterised family of gates like the VQE with a hardware efficient ansatz or larger depth QAOA which cannot be plotted in a two-dimensional contour.

V. METHODS

One common metric used, especially in QAOA, is the approximation ratio as given in Eq. (A4). However, as we noted earlier, the true aim of variational quantum algorithms for combinatorial optimisation is to obtain quickly a sufficiently high (but not necessarily close to unity) overlap with the optimal solution. The CVaR method, for example, is constructed in a way that the maximum overlap achieved is not unity but determined by the risk α. While our approach does achieve high approximation ratio, to make a fair and more complete comparison with prior works and importantly with [11], we use different metrics. Specifically, to benchmark and test our proposed method, we used three different types of metrics. The first is the overlap with the optimal solution. If $|\psi_{opt,i}\rangle$ is a d-degenerate ground state of the problem Hamiltonian, then the overlap is defined as:

$$\sum_{i=1}^{d} |\langle\psi(\theta)|\psi_{opt,i}\rangle|^2 \tag{23}$$

i.e. the probability of obtaining the optimal solution, given the parameters θ. It follows that the parameterised state with the highest overlap with the optimal solution leads to sampling that optimal solution with the least number of circuit executions.

The second metric we want to test is the time taken to reach a given fixed overlap. We set a threshold of 10% probability of obtaining the optimal solution and we tested which method achieves at least that probability faster. We note however that, in order to test which method converges to a 10% overlap faster, we have to use $\alpha \geq 0.1$ because all $\alpha < 0.1$ are not guaranteed to converge in an overlap of 10% since the parameters θ than minimise α lead in an overlap smaller than 0.1.

To summarise the results and compare better the different approaches, for each cost function we divided the problem instances to those that the cost function is successful and to those that it fails. The meaning of what constitutes a "successful" run or a "failed" run cannot be unambiguously defined. For our work we consider that an optimiser is successful at a given instance of a problem if it achieves at least 10% overlap with the optimal solution. It is clear that as the size of the problem instances increase, achieving a fixed 10% overlap becomes harder[7]. In our analysis we chose 10% since this leads to interesting behaviour where the methods analysed differ in their performance.

In our experiment, for comparing with fixed α we used four different choices: $\alpha = 0.1, 0.2, 0.5, 1$. The $\alpha = 1$ choice corresponds to a non CVaR objective function. Specifically, $\alpha = 1$ refers to the expectation value (it includes all the measurement outcomes) and it is the objective function that has been used in the overwhelming majority of the existing literature on variational quantum algorithms. In [11] they did an extensive comparison of CVaR with the expectation value (on the same combinatorial problems we make our analysis). For that reason, we choose to make the comparison of all different choices of α with our proposed α_t and plot our results in one section (see Section VI).

We also note that ascending factors $\lambda \in [0.025, 0.045]$ and $\lambda \in [0.3, 0.4]$ were found to be a good choice for

[7] We should note that even a much smaller overlap is sufficient to find at least once the solution, provided that the number of "shots" is sufficiently large.

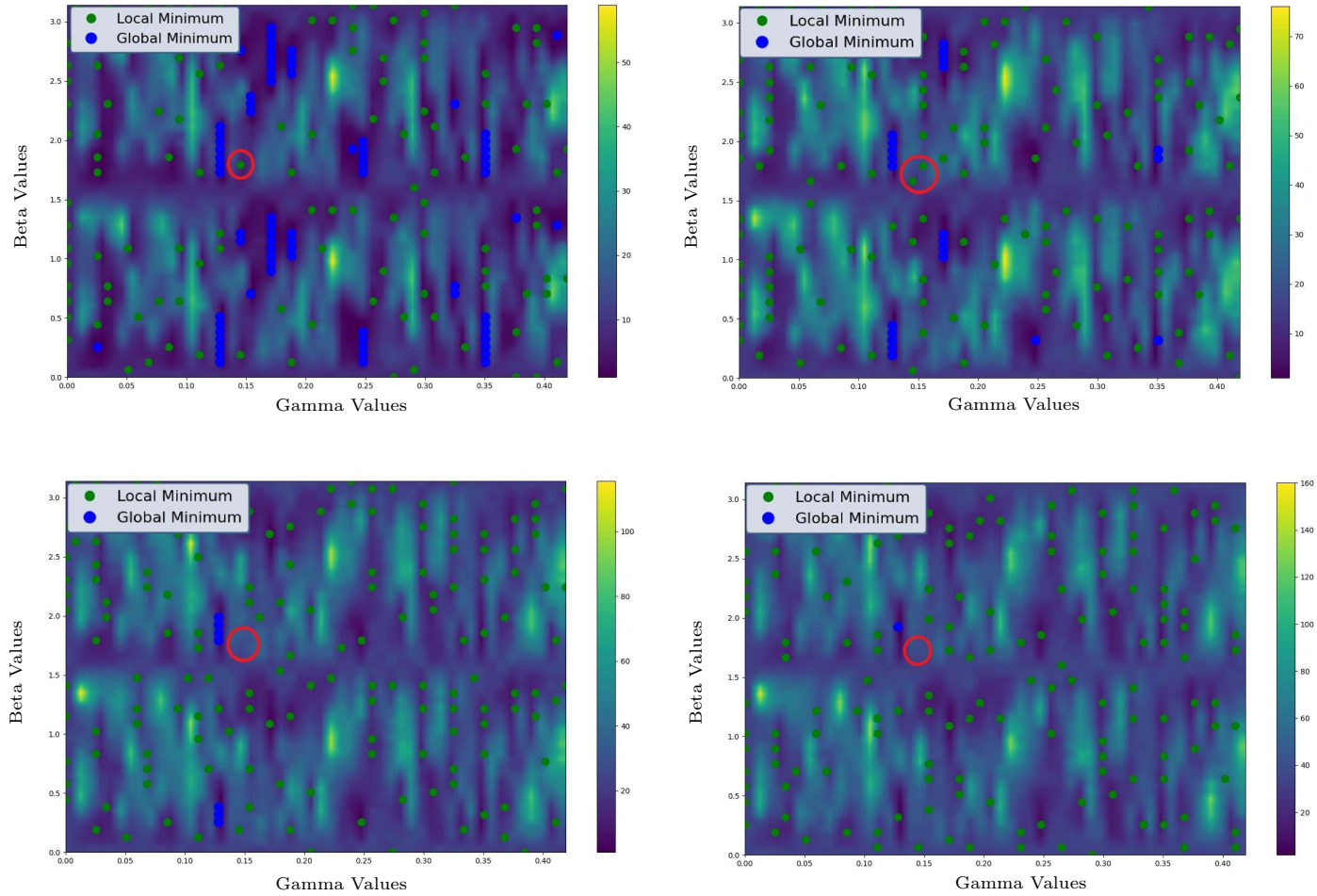

Figure 4: Visualisation of local and global minima for different $CVaR_\alpha$ objective functions. On the top two figures, corresponding to $\alpha = 0.05$ on the left and $\alpha = 0.08$ on the right, you can see the local minima drawn in the red circle. However, on the bottom figures, corresponding to $\alpha = 0.11$ on the left and $\alpha = 0.14$ on the right, the local minima no longer exist.

the three different problems on instances with 15 to 20 qubits for the linear and sigmoid ascending respectively. (see Appendix C). However, we would like to stress that if the sizes of the instances increase or even if the problems change but the sizes remain the same, one would have to readjust the hyperparameter λ. Investigating theoretically the choice of both the ascending factor and the ascending function given the characteristics of the problem as well as possible connections of our method to adiabatic quantum computing goes beyond the scope of this paper but will be investigated in a subsequent work.

In the QAOA algorithm we tested instances using depth $p = 1$ to $p = 6$ while on VQE we worked only on the depth $p = 1$, since this depth was sufficient to get very good accuracy. In near term devices for the QAOA algorithm, increasing the depth even more becomes impractical due noise and decoherence. For this reason we did not consider greater depth, despite the fact that theoretically this could lead to better performance. This means that the variational ansatz for QAOA has only 2 to 12 parameters, i.e. only a fraction of the total parameters present in hardware efficient ansatz used for VQE in depth-1.

To account for the different sizes of problem instances, and to make a fair comparison for the speed of convergence, we used the normalised optimiser iterations [31]. Note that this choice is made in order to be able to compare the performance of the algorithm among instances that involve different number of qubits, and see how the improvement offered by Ascending-CVaR is independent of the instance size. Concretely, the normalised optimiser iterations is defined as the number of times the optimiser evaluates the objective function divided by the function's number of parameters, i.e. the number of parameters of the ansatz. In the

case of the VQE the number of parameters are $n(1+p)$ while on QAOA are $2p$. We note however, that the real time of convergence could be used as seen in Appendix B, where we compare the performance with respect to the total number of circuit repetitions. However, as we show below, there are instances where the constant CVaR does not achieve even a small overlap with the optimal solution and in those cases the time taken becomes irrelevant.

We ran our experiments on IBM's *Qiskit Aer* simulator, allowing noiseless multi-shot executions of our circuit. We set the number of executions of our circuit to $K = 1000$, which scaled up as K/α with the choice of α. All instances were given a maximum of $(66 \times parameters)$ optimiser iterations which is more than enough iterations for an optimiser to converge to a minimum in the problems we implemented. They were initialised with a random choice of parameters, but the same for all different choices of α. We used the same gradient-free optimiser, COBYLA [32], for all different problems and instances as it was shown to outperform other classical optimisers [33].

VI. RESULTS

We will analyse the results for each of the three combinatorial optimisation problems separately. For each of them we will present the results for VQE with hardware-efficient ansatz first and then the results for QAOA. We note that for all three combinatorial optimisation problems and for all methods used (Ascending-CVaR, constant CVaR, expectation value), VQE performs (much) better than QAOA, at least for the sufficiently shallow circuits that we consider. Our method improves the performance in both cases (VQE, QAOA) but since VQE gives much better results for these problems, in the comparison and discussion we will focus on VQE instances only.

A. Max-Cut

For the *Max-Cut* problem we worked on unweighted graphs with 15-19 vertices, drawn from different graph classes and sampled them using the NetworkX library [34].

1. $CVaR_{\alpha_t}$-VQE

For regular graphs, CVaR_{α_t}-VQE behaved equally well with constant-α's optimisation as well as with the expectation value. All of the methods reached the chosen threshold of 10% overlap with the optimal state at almost equal times without any difficulty. For that reason we focused on harder non-regular instances where our method outperformed the latter methods. In Table I we give a summary of the results for 100 random non-regular unweighted graph instances with 15 to 19 vertices. We can see that our method succeeds in more instances while the overlap achieved is also much higher.

There are many reasons why non-regular random graphs are "harder" than regular graphs. The first is that the ground state of a regular graph, due to its symmetry, is highly degenerate where the optimiser could easily reach without "stucking" in a sub-optimal minimum. The second is that the Hamiltonian corresponding to a random graph has more distinct eigenvalues and as it was shown numerically by [33], the number of distinct eigenvalues correlates inversely with the performance of hardware efficient ansatz.

Indicatively, in Figure 5 we plot the probability of sampling the optimal solution over the normalised number of iterations for two random graphs with 17 vertices. For the left figure, we can see how the optimiser for the Ascending-CVaR optimisation is able to to find the optimal solution in under 10 normalised iterations which by the end of the optimisation is able to increase the probability up to 70%. Notably the expectation value or constant CVaR completely fail. The right part of the figure, gives another example where our approach performs better. This instance constitutes an example where smaller α's do not lead to better performance for constant CVaR[8]. We can see in the figure that while $\alpha = 0.1$ failed, $\alpha = 0.2$ was able to achieve a high quality parameterised state. This is another indication why our approach is more flexible.

2. $CVaR_{\alpha_t}$-QAOA

Solving the Max-Cut problem using QAOA, with small depth circuits, does not seem a very promising approach in any of the methods considered (constant CVaR or Ascending-CVaR). In terms of speed, all methods converged equally fast but in states with small overlap with the solution (with relatively small differences within different approaches). Having said that, as explained below, our method still gives improved performance.

While CVaR_{α_t}-VQE optimisation results in high overlap states, CVaR_{α_t}-QAOA produces "flat" states, a behaviour also observed in [11]. These states have almost equal probability amplitudes to the majority of

[8] In most cases, small α gives better performance, but one cannot know a-priori which is the suitable α in the constant CVaR case.

Max-Cut	Successful Instances					Average Overlap				
	α_t	0.1	0.2	0.5	1	α_t	0.1	0.2	0.5	1
Random Graphs	96	84	81	68	53	64.69	12.13	21.45	39.28	36.24

Table I: Results table for the *Max-Cut* problem (VQE) for 100 random non-regular unweighted graph instances with 15 to 19 vertices.

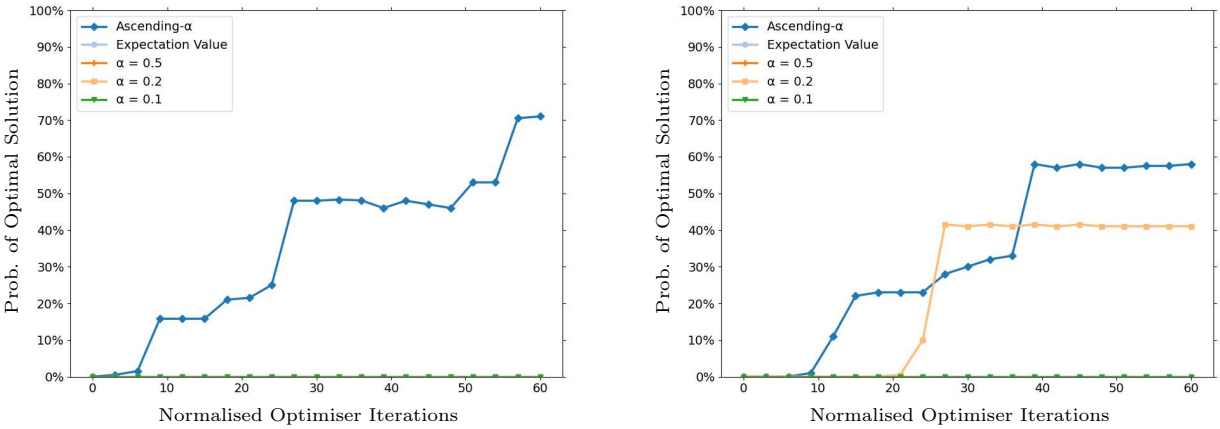

Figure 5: *Max-Cut* instances with 17 vertices for random non-regular unweighted graphs. *Ascending-CVaR*, drawn with a blue line (diamond marker), results in a fast and high overlap with the optimal solution in contrast to constant CVaR.

the computational basis states. For the Max-Cut problem, as noted in [17], it seems that the states produced with QAOA with small p result to states with energy close to the (random) initialisation point. The spread of the energies does increase with p, possibly leading to a state close to the ground state, but in our analysis we focused on small $p \leq 6$. Intuitively, the main reason why QAOA cannot achieve the same probability amplitudes as VQE, in the same depth, is due to having a smaller number of parameters as well as the architecture of the ansatz [35].

Note that the parameter space is filled with sub-optimal local minima. Constant CVaR objective functions with different confidence level α's lead to different energy landscape. This means that a local minimum for a confidence level α_1 does not, in general, correspond to a local minimum for a confidence level α_2 if $\alpha_1 \neq \alpha_2$. This is probably the reason that we get improved performance. For example, in Figure 6 we see how Ascending-CVaR can avoid local minima. In this example all constant CVaR achieve less than 3% overlap with the ground state, while the Ascending-CVaR gives 7%.

B. Number Partitioning

On *Number Partitioning* we tested instances with 17 to 20 integers, on both VQE and QAOA.

1. *CVaR$_{\alpha_t}$-VQE*

On CVaR$_{\alpha_t}$-VQE we tested 300 instances with 17 to 20 integers, sampled randomly from three sets; $N_1 = \{0, \ldots, 200\}$, $N_2 = \{0, \ldots, 500\}$ and $N_3 = \{0, \ldots, 750\}$. We highlight that the smaller the set that the numbers are uniformly drawn from, the easier the optimiser succeeds in finding the optimal solution. A summary of the results is given at Table II.

For the first two sets, we used a linear ascending function with an ascending factor $\lambda = 0.03$. Further optimisation of the parameter may lead in either faster convergence or more successful instances. Either way, the Ascending-CVaR method outperforms constant CVaR and the expectation value objective function on the aforementioned metrics (e.g. see typical performance on Figure 7).

For the last set N_3, constant CVaR and the expectation value as objective functions struggled to achieve even a small overlap with the optimal solution. Indica-

NP	Successful Instances					Average Overlap				
	α_t	0.1	0.2	0.5	1	α_t	0.1	0.2	0.5	1
N_1	87	85	66	16	2	54.17	11.50	16.56	7.94	0.99
N_2	80	69	29	11	0	48.33	10.24	7.56	5.88	0.4
N_3^*	95	58	24	9	0	56.85	8.24	5.84	3.45	0.16

Table II: Results table for the *Number Partitioning* problem (VQE) for the three different sets N_1, N_2 and N_3^*, where the star at the last set indicates that we used the sigmoid ascending function.

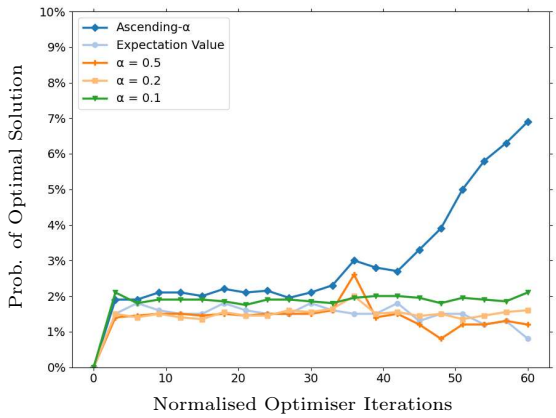

Figure 6: CVaR$_{\alpha_t}$-QAOA optimisation with linear ascending for a Max-Cut instance of 17 qubits. The blue line (diamond marker), indicating the ascending optimisation, results in more than a 100% increase in the overlap with the optimal solution in contrast to the expectation value or constant CVaR optimisation.

tively, at 40% of the cases none of the constant CVaR objective functions could be "successful"[9]. We found that by choosing a sigmoid ascending function, the optimiser is able to attain a high quality parameterised state and succeed in the majority of instances (95%). The trade-off is that using the sigmoid ascending function, in contrast to linear ascending, comes with some cost of more circuit shots in order to achieve the same accuracy. Note also, that the linear ascending function, while performing worse than the sigmoid, it was still more successful than the constant CVaR objective functions.

[9] Recall, that successful in our convention, means to achieve overlap of at least 10% with the optimal solution.

2. *CVaR$_{\alpha_t}$-QAOA*

While CVaR$_{\alpha_t}$-VQE optimisation efficiently achieved a high overlap state already within the first layer for instances drawn from the two sets N_1 and N_2, CVaR$_{\alpha_t}$-QAOA failed to achieve a high overlap on small depths. To address this issue without having to increase the depth of the ansatz we chose to work on instances drawn from the smaller set $M = \{0, \ldots, 50\}$. For the Number Partitioning problem, the cost function's parameter space is highly dependent on the set we draw the numbers from. The unitary transformation $e^{i\gamma H_C}$ is composed of $e^{i\gamma n_k n_l \sigma_z^k \sigma_z^l}$ terms where n_k, n_l correspond to the numbers on the k and l index respectively. The parameter γ is then restricted to $0 \leq \gamma < 2\pi/(n_j n_m)$ with n_j and n_m corresponding to the two smallest numbers of the set.

Our method succeeds in finding quantum states with higher overlap, unreachable with constant CVaR optimisation, possibly because it avoids the high amount of local minima. Indicatively in Figure 8 we see an example where Ascending-CVaR achieves more than double overlap with the optimal solution than other methods, but is still below the threshold of 10% required to classify this as a "successful run".

C. Portfolio Optimisation

On *Portfolio Optimisation* we tested instances with 16 to 20 assets, on both VQE and QAOA, with a budget drawn uniformly at random from the set $B = \{0, ...n\}$ where n is the number of assets and many different risk factors q.

1. *CVaR$_{\alpha_t}$-VQE*

We used linear ascending with an ascending factor $\lambda = 0.045$ and the confidence level was initialised on $\alpha_0 = 0.01$. The results are summarised in Table III. In Figure 9 we see the typical performance of two different instances where we plotted the probability of obtaining the optimal solution over the normalised number of optimiser iterations for the CVaR$_{\alpha_t}$-VQE.

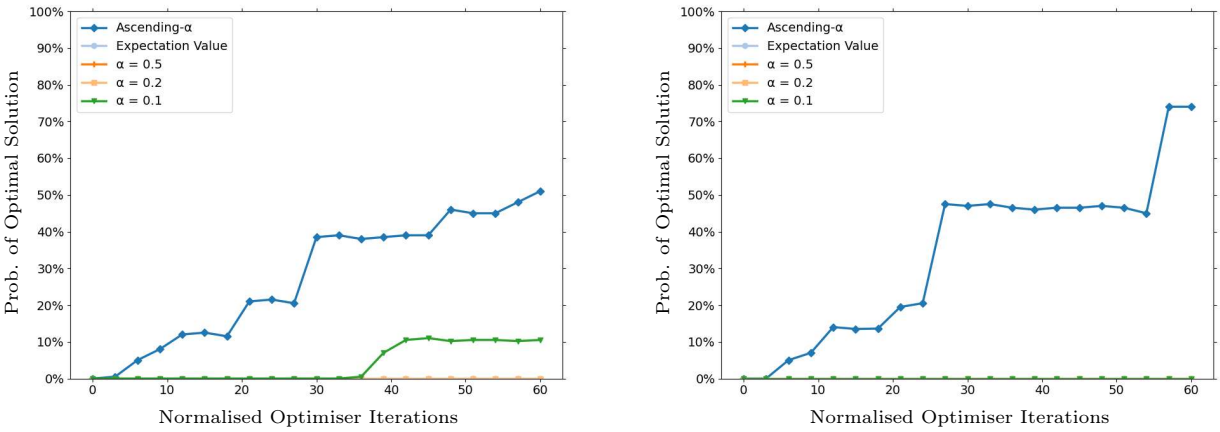

Figure 7: Probability of sampling the optimal solution for *Number Partitioning* instances with 17-20 integers uniformly drawn from the sets $N_1 = \{0, \ldots, 200\}$ (on the left) and $N_2 = \{0, \ldots, 500\}$ (on the right). The blue line (diamond marker), indicating *Ascending-CVaR* outperforms constant CVaR in terms of speed and overlap with the optimal solution.

Portfolio Optimisation	Successful Instances					Average Overlap				
	α_t	0.1	0.2	0.5	1	α_t	0.1	0.2	0.5	1
Random Portfolios	100	100	100	16	1	63.25	13.35	24.74	9.42	0.64

Table III: Results table for the *Portfolio Optimisation* problem (VQE) for 100 random Portfolios with 16 to 20 assets.

We highlight the fact that Ascending-CVaR and constant CVaR with $\alpha = 0.1, 0.2$ succeed in achieving at least 10% overlap on all instances tested (see results on Table III), while the expectation value ($\alpha = 1$) failed in almost all cases. Moreover, it is worth noting that our method offers a significant improvement in comparison with all the other approaches in the speed that this overlap was achieved (in terms of normalised optimiser iterations and circuit repetitions) and in the overall magnitude of the overlap achieved (see also Table III).

2. *CVaR$_{\alpha_t}$-QAOA*

CVaR$_{\alpha_t}$-QAOA, similarly with [11], underperforms significantly in terms of overlap with the optimal state, compared to CVaR$_{\alpha_t}$-VQE. Specifically, keeping the depth as in previous parts, and without increasing the shots each circuit is implemented, all methods fail achieving overlap with the optimal solution well below 1%. There are several reasons for this failure, including the *Reachability Deficits* [36], the large problem density [33] and *Barren Plateaus* [37]. This, however, goes beyond the focus of this paper that is to find a way to improve the performance of previously used objective functions. To illustrate the improvement, we could have used (significantly) larger number of shots, where Ascending-CVaR would start showing better performance. This would make the comparison with other problems unfair (where in all cases we used the same "normalised" number of shots), and it would still not present a practical way to solve the Portfolio Optimisation problem (VQE is much better), so we omitted it.

VII. CONCLUSIONS

We introduced a novel type of objective function, Ascending-CVaR, to be used in variational quantum algorithms for any combinatorial optimisation problem. The starting point is the (constant) CVaR objective function of [11], where they illustrated that for any choice of risk α the true ground state is a minimum, and that with (typically small values of) α one can improve the performance compared to the "standard" expectation-value objective function. Our idea was to use an evolving objective function that "passes" through all the different values of α to finish at the expectation

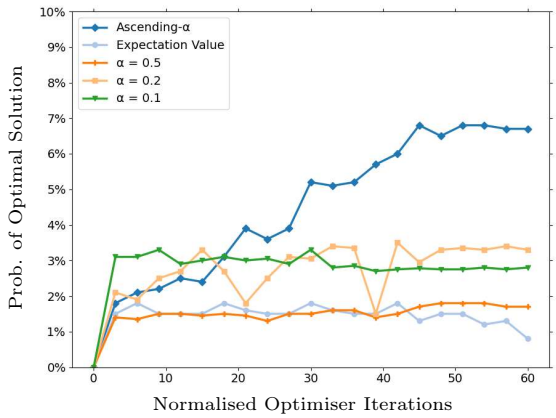

Figure 8: CVaR_{α_t}-QAOA for an 18-integer instance Number Partitioning problem with $p = 4$. The blue line (diamond marker), indicating Ascending-CVaR optimisation, is able to achieve 100% increase in the overlap with the optimal solution in respect to the other objective functions.

value. This, intuitively, avoids getting stuck at local minima since the energy landscape for different α's differs apart from the global minimum.

We tested numerically the proposal on three combinatorial optimisation problems (Max-Cut, Number Partitioning and Portfolio Optimisation), where in agreement with prior works we found that for these problems VQE seems more promising than QAOA with small depth. The improvement that Ascending-CVaR provides to VQE and QAOA are similar but we focus on VQE here since this was the overall more promising approach to solve the corresponding optimisation problems.

We observed that Ascending-CVaR gave much greater on average overlap with the optimal solution (see Table IV). In Portfolio Optimisation and Number Partitioning we got 10 times greater overlap than the expectation value (while we got at least double overlap than the best constant CVaR choice). In Max-Cut we got smaller improvement (80%) compared to the expectation value, but note that the constant CVaR actually gave much smaller overlap. Perhaps the most important feature is that in the Number Partitioning and Max-Cut, Ascending-CVaR succeeded in finding the solution in many instances that no other approach achieved more than the small chosen threshold of 10% overlap. This indicates that not only the approach improves the quality of the results, but is plausible that instances that are believed to be "hard" with the other methods, will become "easy" and thus solvable.

Beyond the accuracy of the result, another factor to evaluate the performance of variational quantum algorithms is the speed, that can be counted with respect to the (average) number of iterations the optimisation needs to run until the algorithm outputs a (candidate) solution. Since our proposal "passes" through several choices of α, one could expect that the "trade-off" for better overlap would be slower speed and thus more optimisation iterations. Interestingly, not only we do not get any cost in speed, but in most cases we see an improvement, i.e. our method requires fewer iterations to reach the threshold of 10% overlap with the solution (see Table V). The only case that our method required slightly more iterations than the $\alpha = 0.1$ was for the case that we actually observed the greater improvement in overlap. This was the Number Partitioning from the set N_3, where the overlap was seven times better than the "next best" case, and 350 times greater overlap than the expectation value (see Table II).

Our work, not only offers a generic method to improve the performance of variational quantum algorithms for combinatorial optimisation problems, it also suggests a new direction of research where dynamic objective functions can be used to boost the performance in terms of quality and speed of near-term quantum algorithms. An immediate follow up to the proposal suggested here is to generalise our approach. Concretely, our method introduces two extra degrees of freedom. The hyper-parameter λ and the function according to which the parameter α increases. It is worth exploring a more systematic rule on how to fix these degrees of freedom according to the problem considered and the features of the specific instance. Finally, considering other dynamic objective functions is another direction that is worth pursuing.

The code for the experiments is available at GitHub [10].

ACKNOWLEDGEMENTS

P.W. acknowledges support by the UK Hub in Quantum Computing and Simulation, part of the UK National Quantum Technologies Programme with funding from UKRI EPSRC grant EP/T001062/1 and the Collaborative Computational Project - Quantum Computing (CCP-QC) with funding from UKRI EPSRC grant EP/T026715/1.

[10] `https://github.com/ioankolot/ascending_cvar`

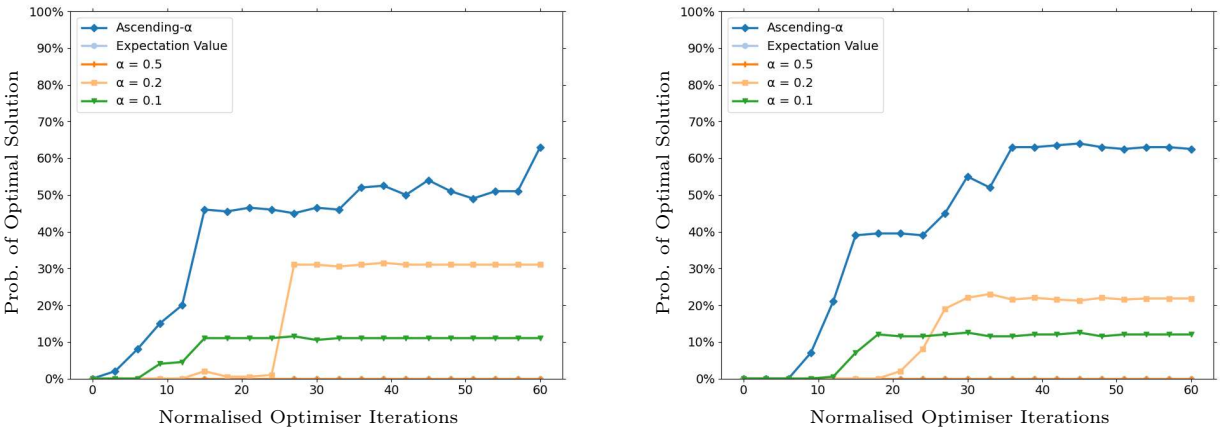

Figure 9: Portfolio Optimisation problem for 18 and 20 asset instances with linear ascending and $\lambda = 0.045$. The blue line (diamond marker) indicates the CVaR$_{\alpha_t}$-VQE optimisation which already within the first 15 optimiser iterations has achieved over 40% overlap, compared to constant α's where either fail ($\alpha = 0.5, 1$), or lead to slower and sub-optimal convergence ($\alpha = 0.1, 0.2$).

Problem Type	Successful Instances %				Overlap Improvement %		
	α_t	0.1	0.2	Expectation Value	α_t	0.1	0.2
Portfolio Optimisation	100	100	100	1	9782	2125	4023
Number Partitioning	87.3	70	39.6	0.6	10225	1858	1464
Max-Cut	96	84	81	53	78.5	-66.52	-40.8

Table IV: Overview of our method

[1] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

[2] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando GSL Brandao, David A Buell, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, 2019.

[3] Daniel J Egger, Jakub Marecek, and Stefan Woerner. Warm-starting quantum optimization. *arXiv preprint arXiv:2009.10095*, 2020.

[4] Sergey Bravyi, Alexander Kliesch, Robert Koenig, and Eugene Tang. Obstacles to state preparation and variational optimization from symmetry protection. *arXiv preprint arXiv:1910.08980*, 2019.

[5] Andrea Skolik, Jarrod R McClean, Masoud Mohseni, Patrick van der Smagt, and Martin Leib. Layerwise learning for quantum neural networks. *Quantum Machine Intelligence*, 3(1):1–11, 2021.

[6] Ken M Nakanishi, Keisuke Fujii, and Synge Todo. Sequential minimal optimization for quantum-classical hybrid algorithms. *Physical Review Research*, 2(4):043158, 2020.

[7] Matteo M Wauters, Emanuele Panizon, Glen B Mbeng, and Giuseppe E Santoro. Reinforcement learning assisted quantum optimization. *arXiv preprint arXiv:2004.12323*, 2020.

[8] Li Li, Minjie Fan, Marc Coram, Patrick Riley, Stefan Leichenauer, et al. Quantum optimization with a novel gibbs objective function and ansatz architecture search. *Physical Review Research*, 2(2):023074, 2020.

[9] Marcello Benedetti, Delfina Garcia-Pintos, Oscar Perdomo, Vicente Leyton-Ortega, Yunseong Nam, and Alejandro Perdomo-Ortiz. A generative modeling approach for benchmarking and training shallow quantum circuits. *npj Quantum Information*, 5(1):1–9, 2019.

[10] Song Cheng, Jing Chen, and Lei Wang. Information perspective to probabilistic modeling: Boltzmann machines versus born machines. *Entropy*, 20(8):583, 2018.

[11] Panagiotis Kl Barkoutsos, Giacomo Nannicini, Anton Robert, Ivano Tavernelli, and Stefan Woerner. Improving variational quantum optimization using cvar. *Quantum*, 4:256, 2020.

[12] Tameem Albash and Daniel A Lidar. Adiabatic quantum computation. *Reviews of Modern Physics*, 90(1):015002, 2018.

Problem Type	Instance Class	Average Normalised Iterations		
		α_t	0.1	0.2
Portfolio Optimisation	Random Portfolios	9.64	11.13	16.29
Number Partitioning	N_1	12.1	19.76	25.09
	N_2	14.73	24.13	28.86
	N_3^*	27.12	25.12	33.62
Max-Cut	Random Graphs	8.75	9.3	10.8

Table V: Average Normalised Optimiser Iterations to achieve at least 10% overlap with the optimal solution for the three different combinatorial problems.

[13] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, et al. Variational quantum algorithms. *arXiv preprint arXiv:2012.09265*, 2020.

[14] Stefan H. Sack and Maksym Serbyn. Quantum annealing initialization of the quantum approximate optimization algorithm. *Quantum*, 5:491, July 2021.

[15] Fernando GSL Brandao, Michael Broughton, Edward Farhi, Sam Gutmann, and Hartmut Neven. For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances. *arXiv preprint arXiv:1812.04170*, 2018.

[16] Andrew Lucas. Ising formulations of many np problems. *Frontiers in Physics*, 2:5, 2014.

[17] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[18] Zhihui Wang, Stuart Hadfield, Zhang Jiang, and Eleanor G Rieffel. Quantum approximate optimization algorithm for maxcut: A fermionic view. *Physical Review A*, 97(2):022304, 2018.

[19] Gavin E Crooks. Performance of the quantum approximate optimization algorithm on the maximum cut problem. *arXiv preprint arXiv:1811.08419*, 2018.

[20] Charles Moussa, Henri Calandra, and Vedran Dunjko. To quantum or not to quantum: towards algorithm selection in near-term quantum optimization. *Quantum Science and Technology*, 5(4):044009, 2020.

[21] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Leo Zhou. The quantum approximate optimization algorithm and the sherrington-kirkpatrick model at infinite size. *arXiv preprint arXiv:1910.08187*, 2019.

[22] Stephan Mertens. Number partitioning. *Computational Complexity and Statistical Physics*, page 125, 2006.

[23] Richard E Korf. Multi-way number partitioning. In *IJCAI*, volume 9, pages 538–543, 2009.

[24] Roman Orus, Samuel Mugel, and Enrique Lizaso. Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4:100028, 2019.

[25] Davide Venturelli and Alexei Kondratyev. Reverse quantum annealing approach to portfolio optimization problems. *Quantum Machine Intelligence*, 1(1):17–30, 2019.

[26] Hans Kellerer, Renata Mansini, and M Grazia Speranza. Selecting portfolios with fixed costs and minimum transaction lots. *Annals of Operations Research*, 99(1):287–304, 2000.

[27] Daniel J Egger, Claudio Gambella, Jakub Marecek, Scott McFaddin, Martin Mevissen, Rudy Raymond, Andrea Simonetto, Stefan Woerner, and Elena Yndurain. Quantum computing for finance: state of the art and future prospects. *IEEE Transactions on Quantum Engineering*, 2020.

[28] Jeffrey Cohen, Alex Khan, and Clark Alexander. Portfolio optimization of 40 stocks using the dwave quantum annealer. *arXiv preprint arXiv:2007.01430*, 2020.

[29] N. Slate, E. Matwiejew, S. Marsh, and J. B. Wang. Quantum walk-based portfolio optimisation. *Quantum*, 5:513, July 2021.

[30] Samuel Mugel, Carlos Kuchkovsky, Escolastico Sanchez, Samuel Fernandez-Lorenzo, Jorge Luis-Hita, Enrique Lizaso, and Roman Orus. Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks. *arXiv preprint arXiv:2007.00017*, 2020.

[31] Jorge J Moré and Stefan M Wild. Benchmarking derivative-free optimization algorithms. *SIAM Journal on Optimization*, 20(1):172–191, 2009.

[32] Michael JD Powell. A direct search optimization method that models the objective and constraint functions by linear interpolation. In *Advances in optimization and numerical analysis*, pages 51–67. Springer, 1994.

[33] Giacomo Nannicini. Performance of hybrid quantum-classical variational heuristics for combinatorial optimization. *Physical Review E*, 99(1):013304, 2019.

[34] Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.

[35] Stuart Hadfield, Zhihui Wang, Bryan O'Gorman, Eleanor G Rieffel, Davide Venturelli, and Rupak Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.

[36] V. Akshay, H. Philathong, I. Zacharov, and J. Biamonte. Reachability Deficits in Quantum Approximate Optimization of Graph Problems. *Quantum*, 5:532, August 2021.

[37] Jarrod R McClean, Sergio Boixo, Vadim N Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training land-

scapes. *Nature communications*, 9(1):1–6, 2018.
[38] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. A variational eigenvalue solver on a photonic quantum processor. *Nature communications*, 5(1):1–7, 2014.
[39] Nikolaj Moll, Panagiotis Barkoutsos, Lev S Bishop, Jerry M Chow, Andrew Cross, Daniel J Egger, Stefan Filipp, Andreas Fuhrer, Jay M Gambetta, Marc Ganzhorn, et al. Quantum optimization using variational algorithms on near-term quantum devices. *Quantum Science and Technology*, 3(3):030503, 2018.
[40] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature*, 549(7671):242–246, 2017.
[41] Joonho Lee, William J Huggins, Martin Head-Gordon, and K Birgitta Whaley. Generalized unitary coupled cluster wave functions for quantum computation. *Journal of chemical theory and computation*, 15(1):311–324, 2018.
[42] Dave Wecker, Matthew Hastings, and Matthias Troyer. Towards practical quantum variational algorithms. 2018.
[43] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D Lukin. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *Physical Review X*, 10(2):021067, 2020.
[44] Lennart Bittel and Martin Kliesch. Training variational quantum algorithms is np-hard–even for logarithmically many qubits and free fermionic systems. *arXiv preprint arXiv:2101.07267*, 2021.
[45] Ruslan Shaydulin, Ilya Safro, and Jeffrey Larson. Multi-start methods for quantum approximate optimization. In *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–8. IEEE, 2019.

Appendix A: Variational Quantum Algorithms

In this section we introduce the details of the two main variational quantum algorithms used in this paper.

a. *Variational Quantum Eigensolver*

The Variational Quantum Eigensolver, as proposed by [38], is a hybrid quantum-classical algorithm, originally designed to solve quantum chemistry problems, but it can be used to tackle optimisation problems [33]. The main idea is to a map the optimisation problem into a cost function that is translated into an interacting qubit Hamiltonian [39], whose ground state corresponds to the solution of the optimisation problem.

The encoded qubit Hamiltonian, H_C, is decomposed into a linear combination of Pauli strings P_a, consisted of tensor products of Pauli Matrices $\hat{\sigma}^x, \hat{\sigma}^y, \hat{\sigma}^z$:

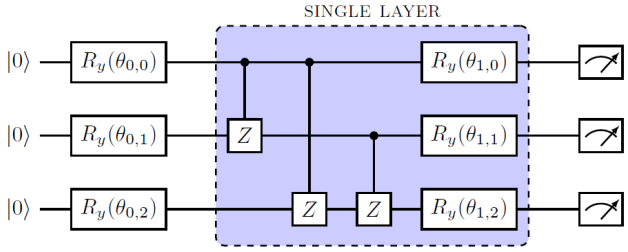

Figure 10: Single Layer Hardware Efficient Ansatz for 3 qubits.

$$H_C = \sum_{k=1}^{M} c_k P_k \tag{A1}$$

where $M = \mathcal{O}(poly(n))$, n is the system size and c_k is the complex coefficient of the P_k Pauli string. However, for combinatorial optimisation problems where the Hamiltonian is a diagonal matrix, H_C is decomposed only on Pauli strings consisting of σ_i^z operators.

VQE is initialised by creating a parameterised state $|\psi(\boldsymbol{\theta})\rangle$ whose parameters are iteratively updated by a classical optimiser so as to minimize an objective function, usually the expectation value of (A1). The parameterised state is created by choosing a variational form $U(\boldsymbol{\theta})$ which acts on the initial state $|0\rangle^{\otimes n}$ and produces $|\psi(\boldsymbol{\theta})\rangle$.

Our choice of variational form $U(\boldsymbol{\theta})$ is a *hardware efficient ansatz* [11, 40], where the qubits are initialised in the $|0\rangle$ state and $R_y(\theta_i)$-rotations are applied in each qubit along with control-Z operators. Each layer of the variational form consists of $(CZ)_{ij}$ operations with i the control qubit and j the target qubit, as long as the condition $i < j$ holds, and $R_y(\theta_i)$ rotations for every qubit. If p is the number of layers, then the number of parameters is linear, $\mu = n\,(1 + p)$, in the number of qubits and the variational form spans every basis state already within the first layer.

The *hardware efficient ansatz* falls in the more general category of *problem-agnostic ansatze*, meaning that the structure of the ansatz carries no information about the problem itself and is mostly suited for optimisation problems. Other problems use different ansatz families, like the *Unitary Coupled Cluster* which is widely used in chemistry to obtain the ground state of a molecule [41] or the *Variational Hamiltonian Ansatz* which encodes the problem's Hamiltonian [42].

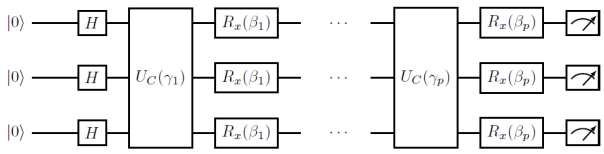

Figure 11: General framework of a p-layer QAOA consisting of $2p$ variational angles.

b. *Quantum Approximate Optimisation Algorithm*

The *Quantum Approximate Optimisation Algorithm* (QAOA) [17] is a variational quantum algorithm mostly used in combinatorial optimisation problems, and while in shallow depths it is analytically and numerically explored for some problems [18, 43], its performance in intermediate depths is still unknown.

The QAOA algorithm applies an alternation of two unitary transformations, one encoding the cost function H_C, $U(H_C) = e^{-i\gamma H_C}$, and the other a mixer Hamiltonian $H_B = \sum \sigma_i^x$, $U(H_B) = e^{-i\beta H_B}$, where γ and β are *variational angles* specifying the "time" for which the unitary transformations are applied. The system is initialised at the ground state of H_B and the alternating ansatz of $U(H_B)\ U(H_C)$ is applied p-times, with p defining the number of layers of the algorithm (see Figure 11), producing the state:

$$|\boldsymbol{\beta}, \boldsymbol{\gamma}\rangle = e^{-i\beta_p H_B} e^{-i\gamma_p H_C} \dots e^{-i\beta_1 H_B} e^{-i\gamma_1 H_C} |+\rangle \tag{A2}$$

where $|+\rangle$ is the uniform superposition state, $\boldsymbol{\gamma} = (\gamma_1 \dots, \gamma_p)$ and $\boldsymbol{\beta} = (\beta_1 \dots, \beta_p)$.

With sufficient repetitions of the algorithm, the expectation value is calculated as:

$$F_p(\boldsymbol{\beta}, \boldsymbol{\gamma}) = \langle \boldsymbol{\beta}, \boldsymbol{\gamma}| H_C |\boldsymbol{\beta}, \boldsymbol{\gamma}\rangle \tag{A3}$$

until the $2p$ optimal parameters $(\boldsymbol{\beta}^*, \boldsymbol{\gamma}^*)$ are found.

If C_{opt} is the optimal cost function, then the target of the algorithm is to maximise the approximation ratio, defined as:

$$r^* = \frac{F_p(\boldsymbol{\beta}^*, \boldsymbol{\gamma}^*)}{C_{opt}} \tag{A4}$$

Finding the optimal parameters is far from trivial since the expectation value landscape is highly non-convex, filled with local minima where a classical optimiser could easily get stuck.

The hardest part of QAOA, and in general of a variational quantum algorithm, is finding the optimal parameters that will lead in a high overlap with the optimal (or near-optimal) bit-string or low expectation value. Recently, [44] proved that training the optimization parameters is NP-Hard and that the landscape of the objective function is filled with far-from-optimal local minima. One way to avoid "getting stuck" in a local minima is using multi-start methods [45] or heuristic methods like using the global optimum of one layer, in QAOA, as a starting point for the next [43].

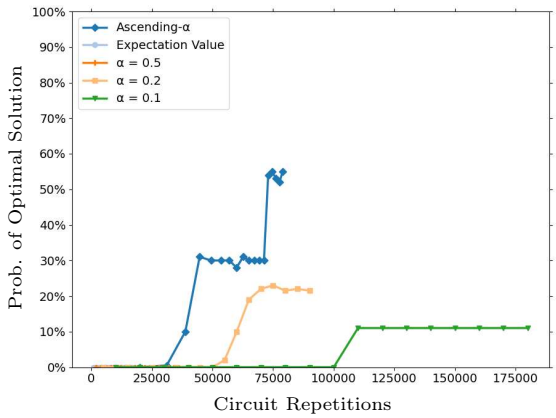

Figure 12: Probability of sampling an optimal solution over the circuit repetitions for a Number-Partitioning instance.

Appendix B: Circuit Repetitions

In this section, we demonstrate how our method outperforms, in terms of real circuit repetitions and quality of the output state, the previously used objective functions. We set our "default" circuit repetitions to $K = 1000$ which we then scale it up, along the discretely increasing α using the expression K/α_t, for each given time. While one may think that this would weaken our results, as illustrated below, it seems that in terms of circuit repetitions our method converges to the chosen threshold of 10% faster than the best of constant CVaR or the expectation value approaches.

Appendix C: Numerical analysis of Ascending factor

In this section we illustrate how the performance of our algorithm depends on the choice of the *ascending factor* λ. We numerically tested a large number of instances of sizes from 16 to 20 qubits and observed that the algorithm performed optimally for ascending factors drawn from the set $[0.025, 0.045]$. For this reason,

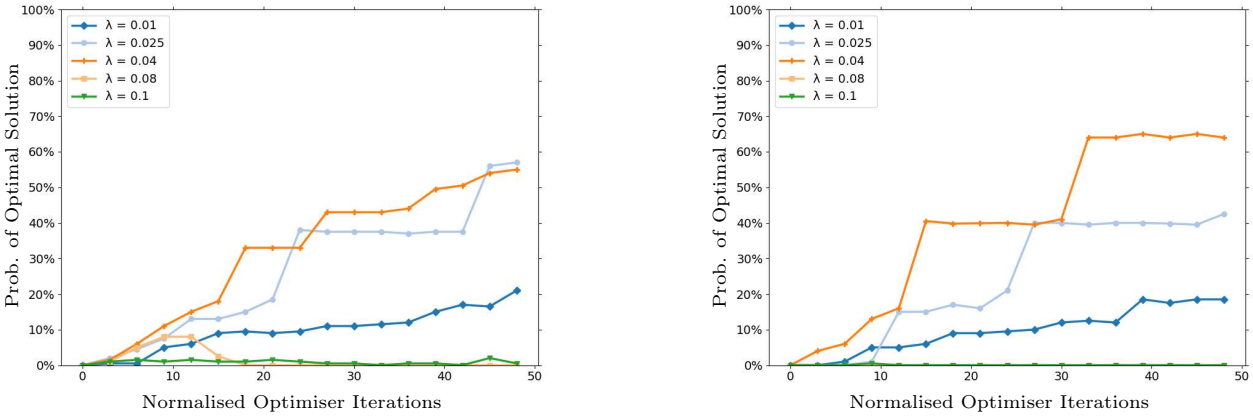

Figure 13: Performance of *Ascending-CVaR* algorithm with linear ascending for different choices of the ascending factor λ. The yellow (square marker) and green (down-pointing marker) lines which refer to $\alpha = 0.08$ and $\alpha = 0.1$ respectively are not able to reach a good approximation to the optimal solution. On the other hand, the orange (line marker) and light blue (circle marker) lines which correspond to $\alpha = 0.04$ and $\alpha = 0.025$ are both able to achieve an overlap larger than 50%. Finally, the dark blue line (diamond marker) is still able to reach a good approximation to the ground state but it lacks in terms of speed of convergence and magnitude of overlap achieved. The graph on the left refers to a Number Partitioning problem while the graph on the right to Portfolio Optimisation Problem.

as an example, we choose to draw the behavior of our algorithm for two random instances (one for Portfolio Optimisation and one for Number Partitioning) for different choices of the hyperparameter λ.

The performance of our method is sensitive to the choice of λ. A small λ still converges to an optimal solution but requires a large number of iterations, compared to λ chosen from the set $[0.025, 0.045]$ which is able to attain a 10% within a small number of iterations. On the other hand, choosing λ to be large (hoping for a faster convergence) fails to achieve even a minor overlap with the optimal solution. A careful tuning of λ is therefore necessary for the optimal performance of the algorithm given the size and class of the problem at hand.